

C++ Standard Library Issues to be moved in Sofia, Jun. 2025

Doc. no. P3742R0

Date: 2025-06-13

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

- [Ready Issues](#)
 - [4198](#). `schedule_from` isn't starting the schedule sender if decay-copying results throws
 - [4202](#). `enable_sender` should be a variable template
 - [4203](#). Constraints on `get_state` functions are incorrect
 - [4204](#). specification of `as_sndr2(Sig)` in `[exec.let]` is incomplete
 - [4205](#). `let_*.transform_env` is specified in terms of the `let_*` sender itself instead of its child
 - [4208](#). Wording needs to ensure that in `connect(sndr, rcvr)` that `rcvr` expression is only evaluated once
 - [4209](#). `default_domain::transform_env` should be returning `FWD-ENV(env)`
- [Tentatively Ready Issues](#)
 - [4188](#). `ostream::sentry` destructor should handle exceptions
 - [4200](#). The `operation_state` concept can be simplified
 - [4201](#). `with-await-transform::await_transform` should not use a deduced return type
 - [4217](#). Clarify `mdspan` layout mapping requirements for `rank == 0`
 - [4222](#). expected constructor from a single value missing a constraint
 - [4224](#). Philox engines should be freestanding
 - [4227](#). Missing `noexcept` operator in `[exec.when.all]`
 - [4231](#). `datapar::chunk<N>` should use `simd-size-type` instead of `size_t`
 - [4232](#). `datapar::resize` does not resize
 - [4233](#). The helper lambda of `std::erase` for `hive` should specify return type as
 - [4234](#). Including `<hive>` doesn't provide `std::begin/end`
 - [4235](#). `cache_latest_view` and `to_input_view` miss `reserve_hint`
 - [4236](#). `chunk_view::outer_iterator::value_type` should provide `reserve_hint`
 - [4239](#). `flat_map`'s transparent comparator no longer works for string literals
 - [4242](#). `ranges::distance` does not work with volatile iterators
 - [4245](#). Operators that interact with `counted_iterator` and `default_sentinel_t` should be `noexcept`
 - [4247](#). Header `<stdbit.h>` is not yet freestanding

Ready Issues

[4198](#). `schedule_from` isn't starting the schedule sender if decay-copying results throws

Section: 33.9.12.5 [\[exec.schedule.from\]](#) Status: [Ready](#) Submitter: Eric Niebler Opened: 2025-02-03 Last modified: 2025-02-13

Priority: 1

View other [active issues](#) in `[exec.schedule.from]`.

View all other [issues](#) in `[exec.schedule.from]`.

Discussion:

Imported from [cplusplus/sender-receiver #304](#).

33.9.12.5 [\[exec.schedule.from\]](#)/p11 specifies `schedule_from`'s completion operation as follows:

```
[<class Tag, class... Args>(auto, auto& state, auto& rcvr, Tag, Args&&... args) noexcept
-> void {
    using result_t = decayed_tuple<Tag, Args...>;
    constexpr bool nothrow = is_nothrow_constructible_v<result_t, Tag, Args...>;

    try {
        state.async_result.template emplace<result_t>(Tag(), std::forward<Args>(args)...);
    } catch (...) {
        if constexpr (!nothrow) {
            set_error(std::move(rcvr), current_exception());
            return;
        }
    }
    start(state.op-state);
};
```

so if emplacing the result tuple throws, `set_error` is immediately called on the downstream receiver. no attempt is made to post the completion to the specified scheduler. this is probably not the right behavior.

Suggested resolution

The right thing, i think, is to catch the exception, emplace the `exception_ptr` into the `async-result` variant, and then start the schedule operation, as shown below:

```
[<class Tag, class... Args>(auto, auto& state, auto& rcvr, Tag, Args&&... args) noexcept
-> void {
    using result_t = decayed_tuple<Tag, Args...>;
    constexpr bool nothrow = is_nothrow_constructible_v<result_t, Tag, Args...>;

    try {
        state.async-result.template emplace<result_t>(Tag(), std::forward<Args>(args)...);
    } catch(...) {
        if constexpr (!nothrow)
            state.async-result.template emplace<tuple<set_error_t, exception_ptr>>(set_error, current_exception());
    }

    start(state.op-state);
}
```

we also need to change how we specify the variant type of `state.async-result`:

Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, env_of_t<Rcvr>>`. Let `as-tuple` be an alias template that transforms a completion signature `Tag(Args...)` into the tuple specialization `decayed_tuple<Tag, Args...>`, such that `as-tuple<Tag(Args...)>` denotes the tuple specialization `decayed_tuple<Tag, Args...>`, and let `is-nothrow-decay-copy-sig` be a variable template such that `is-nothrow-decay-copy-sig<Tag(Args...)>` is a core constant expression of type `bool` and whose value is `true` if the types `Args...` are all nothrow decay-copyable, and `false` otherwise. Let `error-completion` be a pack consisting of the type `set_error_t(exception_ptr)` if `(is-nothrow-decay-copy-sig<Sigs> &&...)` is `false`, and an empty pack otherwise. Then `variant_t` denotes the type variant<monostate, `as-tuple<Sigs>...`, `error-completion...`>, except with duplicate types removed.

[This touches the same text as LWG 4203⁽¹⁾.]

[2025-02-07; Reflector poll]

Set priority to 1 after reflector poll.

[Hagenberg 2025-02-11; LWG]

Direction seems right. Decay-copyable is not a defined term.

[2025-02-12 Tim adds PR]

This also corrects the issue that `nothrow` is currently relying on the unspecified exception specification of tuple's constructor.

[Hagenberg 2025-02-12; move to Ready]

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 33.9.12.5 [\[exec.schedule.from\]](#) p8 as indicated:

-8- Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, env_of_t<Rcvr>>`. Let `as-tuple` be an alias template that transforms a completion signature `Tag(Args...)` into the tuple specialization `decayed_tuple<Tag, Args...>`, such that `as-tuple<Tag(Args...)>` denotes the type `decayed_tuple<Tag, Args...>`, and let `is-nothrow-decay-copy-sig` be a variable template such that `auto(is-nothrow-decay-copy-sig<Tag(Args...)>)` is a constant expression of type `bool` and equal to `(is_nothrow_constructible_v<decay_t<Args>, Args> && ...)`. Let `error-completion` be a pack consisting of the type `set_error_t(exception_ptr)` if `(is-nothrow-decay-copy-sig<Sigs> &&...)` is `false`, and an empty pack otherwise. Then `variant_t` denotes the type variant<monostate, `as-tuple<Sigs>...`, `error-completion...`>, except with duplicate types removed.

2. Modify 33.9.12.5 [\[exec.schedule.from\]](#) p11 as indicated:

- 11- The member `impls-for<schedule_from_t>::complete` is initialized with a callable object equivalent to the following lambda:

```
[<class Tag, class... Args>(auto, auto& state, auto& rcvr, Tag, Args&&... args) noexcept
-> void {
    using result_t = decayed_tuple<Tag, Args...>;
    constexpr bool nothrow = is_nothrow_constructible_v<result_t, Tag, Args...>
                               (is_nothrow_constructible_v<decay_t<Args>, Args> && ...);

    try {
        state.async-result.template emplace<result_t>(Tag(), std::forward<Args>(args)...);
    }
```

```

    } catch (...) {
        if constexpr (!nothrow) {
            state.async_result.template emplace<tuple<set_error_t, exception_ptr>>(set_error, current_exception());
            set_error(std::move(rcvr), current_exception());
            return;
        }
    }
    start(state.op-state);
};

```

4202. ~~enable-sender~~ should be a variable template

Section: 33.9.3 [exec.snd.concepts] Status: [Ready](#) Submitter: Eric Niebler Opened: 2025-02-03 Last modified: 2025-02-12

Priority: 1

Discussion:

Imported from [cplusplus/sender-receiver #305](#) and [cplusplus/sender-receiver #306](#).

We require an opt-in to satisfy the sender concept. Making your type awaitable with an empty environment is one way to opt in. If your awaitable requires an environment, you have two options:

- give your awaitable type a nested sender_concept typedef, or
- specialize `enable_sender` for your awaitable type.

The problem is that somewhere along the way, the `enable_sender` variable template was turned into an exposition-only ~~enable-sender~~ concept. We should roll back that change.

[2025-02-07; Reflector poll]

Set priority to 1 after reflector poll.

[Hagenberg 2025-02-11; move to Ready]

Proposed resolution:

This wording is relative to [N5001](#).

1. Change 33.9.3 [exec.snd.concepts] as indicated:

```

template<class Sndr>
concept is-sender =                                // exposition only
    derived_from<typename Sndr::sender_concept, sender_t>;

template<class Sndr>
concept enable-sender =                            // exposition only
    is-sender<Sndr> ||
    is-awaitable<Sndr, env-promise<empty_env>>;      // [exec.awaitable]

template<class Sndr>
inline constexpr bool enable_sender = enable_sender<Sndr>;

template<class Sndr>
concept sender =
    bool(enable_senderenable_sender<remove_cvref_t<Sndr>>> &&
    requires (const remove_cvref_t<Sndr>& sndr) {
        { get_env(sndr) } -> queryable;
    } &&
    move_constructible<remove_cvref_t<Sndr>> &&
    constructible_from<remove_cvref_t<Sndr>, Sndr>;

...

-2- Given a subexpression sndr, [...]

```

~~[-? - Remarks: Pursuant to 16.4.5.2.1 [namespace.std], users may specialize `enable_sender` to true for cv-unqualified program-defined types that model sender, and false for types that do not. Such specializations shall be usable in constant expressions (7.7 [expr.const]) and have type `const bool`.~~

4203. Constraints on `get-state` functions are incorrect

Section: 33.9.12.5 [exec.schedule.from] Status: [Ready](#) Submitter: Eric Niebler Opened: 2025-02-03 Last modified: 2025-02-12

Priority: 1

View other [active issues](#) in [exec.schedule.from].

View all other [issues](#) in [exec.schedule.from].

Discussion:

Imported from: [cplusplus/sender-receiver #313](#) and [cplusplus/sender-receiver #314](#).

33.9.12.5 [\[exec.schedule.from\]](#) p6 reads:

The member `impls-for<schedule_from_t>::get-state` is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(see below)
  requires sender_in<child-type<Sndr>, env_of_t<Rcvr>> {
```

The constraint should account for the fact that the child sender will be connected with `FWD-ENV(get_env(rcvr))`.

[The resolution touches some of the same text as LWG [4198^{\(i\)}](#), but without conflicting.]

Imported from: [cplusplus/sender-receiver #315](#).

33.9.12.11 [\[exec.when.all\]](#) p6 reads:

The member `impls-for<when_all_t>::get-env` is initialized with a callable object equivalent to the following lambda expression:

```
[]<class State, class Rcvr>(auto&& state, const Receiver& rcvr) noexcept {
  return see below;
}
```

Returns an object `e` such that

- (6.1) — `decltype(e)` models *queryable*, and
- (6.2) — `e.query(get_stop_token)` is expression-equivalent to `state.stop_src.get_token()`, and
- (6.3) — given a query object `q` with type other than `cv stop_token_t`, `e.query(q)` is expression-equivalent to `get_env(rcvr).query(q)`.

The problem is in (6.3). It should be forwarding on *forwarding-query*'s to `get_env(rcvr)` but is instead forwarding all queries.

Imported from: [cplusplus/sender-receiver #316](#).

The child senders should only see the parent's queries if they are forwarding queries.

Imported from: [cplusplus/sender-receiver #311](#).

33.9.12.13 [\[exec.stopped.opt\]](#)/para 3 reads:

Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *sender-for<Sndr, stopped_as_optional_t>* is false, or if the type *single-sender-value-type<Sndr, Env>* is ill-formed or void, then the expression `stopped_as_optional.transform_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

the test for *single-sender-value-type<Sndr, Env>* is incorrect. It should be testing its child for single-sender-ness.

In addition, it should be applying *FWD-ENV-T* to `Env` so that only forwarding queries are forwarded.

[2025-02-07; Reflector poll]

Set priority to 1 after reflector poll.

[Hagenberg 2025-02-11; move to Ready]

Proposed resolution:

This wording is relative to [N5001](#).

1. Change 33.9.2 [\[exec.snd.expos\]](#) as indicated:

-2- For a queryable object `env`, *FWD-ENV(env)* is an expression whose type satisfies *queryable* such that for a query object `q` and a pack of subexpressions `as...`, the expression `FWD-ENV(env).query(q, as...)` is ill-formed if `forwarding_query(q)` is false; otherwise, it is expression-equivalent to `env.query(q, as...)`. *The type `FWD-ENV-T(Env)` is `decltype(FWD-ENV(declval<Env>()))`.*

2. Change 33.9.12.5 [\[exec.schedule.from\]](#) as indicated:

-6- The member `impls-for<schedule_from_t>::get-state` is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(see below)
  requires sender_in<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)> {
```

...

-8- Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)>`. Let *as-tuple* be an alias template that

transforms a completion signature `Tag(Args...)` into the tuple specialization `decayed-tuple<Tag, Args...>`. Then `variant_t` denotes the type `variant<monostate, as-tuple<Sigs>...>`, except with duplicate types removed.

3. Change 33.9.12.8 [\[exec.let\]](#) as indicated:

-6- Let `receiver2` denote the following exposition-only class template:

```
namespace std::execution {
    ...
}
```

Invocation of the function `receiver2::get_env` returns an object `e` such that

(6.1) — `decltype(e)` models *queryable* and

(6.2) — given a query object `q`, the expression `e.query(q)` is expression-equivalent to `env.query(q)` if that expression is valid; otherwise, **if the type of `q` satisfies *forwarding-query*, `e.query(q)` is expression-equivalent to `get_env(rcvr).query(q)`; otherwise, `e.query(q)` is ill-formed.**

-7- `impls-for<decayed-typeof<let-cpo>>::get-state` is initialized with a callable object [...]

-8- Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)>`. Let `LetSigs` be a pack of those types in `Sigs` with a return type of `decayed-typeof<set-cpo>`. Let `as-tuple` be an alias template such that `as-tuple<Tag(Args...)>` denotes the type `decayed-tuple<Args...>`. Then `args_variant_t` denotes the type `variant<monostate, as-tuple<LetSigs>...>` except with duplicate types removed.

4. Change 33.9.12.11 [\[exec.when.all\]](#) as indicated:

-6- The member `impls-for<when_all_t>::get-env` is initialized with a callable object equivalent to the following lambda expression:

```
[]<class State, class Rcvr>(auto&& state, const Receiver& rcvr) noexcept {
    return see below;
}
```

Returns an object `e` such that

(6.1) — `decltype(e)` models *queryable*, and

(6.2) — `e.query(get_stop_token)` is expression-equivalent to `state.stop-src.get_token()`, and

(6.3) — given a query object `q` with type other than `cv stop_token_t` **and whose type satisfies *forwarding-query***, `e.query(q)` is expression-equivalent to `get_env(rcvr).query(q)`.

-7- The member `impls-for<when_all_t>::get-state` is initialized with a callable object equivalent to the following lambda expression:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(e) -> decltype(e) {
    return e;
}
```

where `e` is the expression

```
std::forward<Sndr>(sndr).apply(make-state<Rcvr>())
```

and `make-state` is the following exposition-only class template:

```
template<class Sndr, class Env>
concept max-1-sender-in = sender_in<Sndr, Env> && // exposition only
    (tuple_size_v<value_types_of_t<Sndr, Env, tuple, tuple>> <= 1);

enum class disposition { started, error, stopped }; // exposition only

template<class Rcvr>
struct make-state {
    template<max-1-sender-in<FWD-ENV-T(env_of_t<Rcvr>)>... Sndrs>
```

...

-8- Let `copy_fail` be `exception_ptr` if [...]

-9- The alias `values_tuple` denotes the type

```
tuple<value_types_of_t<Sndrs, FWD-ENV-T(env_of_t<Rcvr>)>, decayed-tuple, optional>...>
```

if that type is well-formed; otherwise, `tuple<>`.

5. Change 33.9.12.12 [\[exec.into.variant\]](#) as indicated:

-5- The member `impls-for<into_variant_t>::get-state` is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept
-> type_identity<value_types_of_t<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)>> {
    return {};
}
```

6. Change 33.9.12.13 [\[exec.stopped.opt\]](#) as indicated:

-3- Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype(sndr)` and `Env` is `decltype(env)`. If `sender-for<Sndr, stopped_as_optional_t>` is false, or if the type `single-sender-value-type<child-type<Sndr>, FWD-ENV-T(Env)>` is ill-formed or void, then the expression `stopped_as_optional.transform_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, __, child] = sndr;
using V = single-sender-value-type<child-type<Sndr>, FWD-ENV-T(Env)>;
```

...

4204. specification of `as-sndr2(Sig)` in [exec.let] is incomplete

Section: 33.9.12.8 [exec.let] **Status:** [Ready](#) **Submitter:** Eric Niebler **Opened:** 2025-02-03 **Last modified:** 2025-02-12

Priority: 1

View other [active issues](#) in [exec.let].

View all other [issues](#) in [exec.let].

Discussion:

33.9.12.8 [exec.let]/p9 reads:

Given a type `Tag` and a pack `Args`, let `as-sndr2` be an alias template such that `as-sndr2<Tag(Args...)>` denotes the type `call-result-t<Fn, decay_t<Args>&...>`. Then `ops2_variant_t` denotes the type

```
variant<monostate, connect_result_t<as-sndr2<LetSigs>, receiver2<Rcvr, Env>>...>
```

except with duplicate types removed.

The type `Env` is not specified. It should be `env_t` from paragraph 7.

Paragraphs 8, 9, and 10 only make sense in relation to the lambda in paragraph 7, but that is not at all clear from the current wording. I suggest making paragraphs 8, 9, and 10 sub-bullets of paragraph 7.

[2025-02-07; Reflector poll]

Set priority to 1 after reflector poll.

[Hagenberg 2025-02-11; move to Ready]

Dropped the suggestion to nest p8-10 under p7.

Proposed resolution:

This wording is relative to [N5001](#).

1. Change 33.9.12.8 [exec.let] as indicated:

-7- `impls-for<decayed-typeof<let-cpo>>::get-state` is initialized with a callable object equivalent to the following:

```
[<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) requires see below {
    auto& [_, fn, child] = sndr;
    using fn_t = decay_t<decltype(fn)>;
    using env_t = decltype(let-env(child));
    using args_variant_t = see below;
    using ops2_variant_t = see below;

    struct state-type {
        fn_t fn;                // exposition only
        env_t env;              // exposition only
        args_variant_t args;    // exposition only
        ops2_variant_t ops2;    // exposition only
    };
    return state-type{std::forward_like<Sndr>(fn), let-env(child), {}, {}};
}
```

-8- Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, env_of_t<Rcvr>>`. Let `LetSigs` be a pack of those types in `Sigs` with a return type of `decayed-typeof<set-cpo>`. Let `as-tuple` be an alias template such that `as-tuple<Tag(Args...)>` denotes the type `decayed-tuple<Args...>`. Then `args_variant_t` denotes the type `variant<monostate, as-tuple<LetSigs>...>` except with duplicate types removed.

-9- Given a type `Tag` and a pack `Args`, let `as-sndr2` be an alias template such that `as-sndr2<Tag(Args...)>` denotes the type `call-result-t<Fn, decay_t<Args>&...>`. Then `ops2_variant_t` denotes the type

```
variant<monostate, connect_result_t<as-sndr2<LetSigs>, receiver2<Rcvr, Env<env_t>>...>
```

except with duplicate types removed.

-10- The *requires-clause* constraining the above lambda is satisfied if and only if the types `args_variant_t` and `ops2_variant_t` are well-formed.

4205. `let_*[*].transform_env` is specified in terms of the `let_*` sender itself instead of its child

Section: 33.9.12.8 [\[exec.let\]](#) **Status:** [Ready](#) **Submitter:** Eric Niebler **Opened:** 2025-02-04 **Last modified:** 2025-02-12

Priority: 1

View other [active issues](#) in `[exec.let]`.

View all other [issues](#) in `[exec.let]`.

Discussion:

Imported from [cplusplus/sender-receiver #319](#).

33.9.12.8 [\[exec.let\]](#) para 13 reads:

13. Let `snldr` and `env` be subexpressions, and let `Sndr` be `decltype((snldr))`. If *sender-for*<`Sndr`, *decayed-typeof*<`let-cpo`>> is false, then the expression `let-cpo.transform_env(snldr, env)` is ill-formed. Otherwise, it is equal to `JOIN-ENV(let-env(snldr), FWD-ENV(env))`.

The sender passed to `let-env` here should be the *child* of `snldr`.

[2025-02-07; Reflector poll]

Set priority to 1 after reflector poll.

"We seem to be missing a guarantee that `auto [_,_ , child] = snldr;` works. We guarantee that it can be used in a structured binding, but not that it must work with a size of three."

[Hagenberg 2025-02-11; move to Ready]

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 33.9.12.8 [\[exec.let\]](#) as indicated:

-13- Let `snldr` and `env` be subexpressions, and let `Sndr` be `decltype((snldr))`. If *sender-for*<`Sndr`, *decayed-typeof*<`let-cpo`>> is false, then the expression `let-cpo.transform_env(snldr, env)` is ill-formed. Otherwise, it is equal to `JOIN-ENV(let-env(snldr), FWD-ENV(env))`.

```
auto& [_ , _ , child] = snldr;
return JOIN-ENV(let-env(child), FWD-ENV(env));
```

4208. Wording needs to ensure that in `connect(snldr, rcvr)` that `rcvr` expression is only evaluated once

Section: 33.9.10 [\[exec.connect\]](#) **Status:** [Ready](#) **Submitter:** Eric Niebler **Opened:** 2025-02-07 **Last modified:** 2025-02-12

Priority: Not Prioritized

Discussion:

Imported from [cplusplus/sender-receiver #325](#).

The current wording of `connect(snldr, rcvr)` defines the `new_sndr` expression as `transform_sender(decltype(get-domain-late(snldr, get_env(rcvr))) {}, snldr, get_env(rcvr))`.

It then defines `connect(snldr, rcvr)` as expression equivalent to `new_sndr.connect(rcvr)`.

As currently worded, this requires evaluating the `rcvr` expression twice. Note that the first usage in the `new_sndr` expression is unevaluated, but the second usage in `get_env(rcvr)` is evaluated.

I think we need to add an extra sentence at the end of this section saying "Where the expression `rcvr` is only evaluated once." or similar.

[Hagenberg 2025-02-11; move to Ready]

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 33.9.10 [\[exec.connect\]](#) as indicated:

-6- The expression `connect(sndr, rcvr)` is expression-equivalent to:

(6.1) — `new_sndr.connect(rcvr)` if that expression is well-formed.

Mandates: The type of the expression above satisfies `operation_state`.

(6.2) — Otherwise, `connect-awaitable(new_sndr, rcvr)`.

except that `rcvr` is evaluated only once.

Mandates: `sender<Sndr> && receiver<Rcvr>` is true.

4209. `default_domain::transform_env` should be returning *FWD-ENV*(env)

Section: 33.9.5 [\[exec.domain.default\]](#) **Status:** [Ready](#) **Submitter:** Eric Niebler **Opened:** 2025-02-07 **Last modified:** 2025-02-12

Priority: Not Prioritized

Discussion:

Imported from [cplusplus/sender-receiver #168](#).

When writing a generic recursive sender transform, you need to ability to unpack an unknown sender `S` and recursively transform the children.

For that, it can be useful to know the type of the environment that `S` will use when connecting its child senders, which is why `transform_env` exists.

For an environment `E` and a sender `S` with tag `T` child `C`, the expression `default_domain().transform_env(S, E)` should return an environment `E2` that is identical to the environment of the receiver that `S` uses to connect `C`.

`default_domain().transform_env(S, E)` will first check whether `T().transform_env(S, E)` is well-formed. If so, it will return that (e.g. `when_all_t` has a `transform_env` that adds a stop token to the environment).

If `T().transform_env(S, E)` is not well-formed, what should `default_domain::transform_env` do? At present, it returns `E` unmodified.

But 33.9.12.1 [\[exec.adapt.general\]](#) has this:

[unless otherwise specified, when] a parent sender is connected to a receiver `rcvr`, any receiver used to connect a child sender has an associated environment equal to *FWD-ENV*(`get_env(rcvr)`).

So the correct thing for `default_domain::transform_env` to do is to return *FWD-ENV*(`get_env(rcvr)`).

[Hagenberg 2025-02-11; move to Ready]

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 33.9.5 [\[exec.domain.default\]](#) as indicated:

```
template<sender Sndr, queryable Env>
constexpr queryable decltype(auto) transform_env(Sndr&& sndr, Env&& env) noexcept;

-5- Let e be the expression

    tag_of_t<Sndr>().transform_env(std::forward<Sndr>(sndr), std::forward<Env>(env))

if that expression is well-formed; otherwise, static_cast<Env>FWD-ENV(std::forward<Env>(env)).

-6- Mandates: noexcept(e) is true.

-7- Returns: e.
```

Tentatively Ready Issues

4188. `ostream::sentry` destructor should handle exceptions

Section: 31.7.6.2.4 [\[ostream.sentry\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2025-01-14 **Last modified:** 2025-02-07

Priority: Not Prioritized

View other [active issues](#) in `[ostream.sentry]`.

View all other [issues](#) in [ostream.sentry].

Discussion:

LWG [397^{\(i\)}](#) suggested changing 31.7.6.2.4 [\[ostream.sentry\]](#) to say that the ostream::sentry destructor doesn't throw any exceptions. That issue was closed as resolved by LWG [835^{\(i\)}](#) which included the "Throws: Nothing" change to the sentry destructor. However, that part of the resolution never seems to have been applied to the working draft. [N3091](#) mentions applying LWG 835 for [N3090](#) but the destructor change is missing, maybe because the paragraph for the sentry destructor had been renumbered from p17 to p4 and LWG 835 didn't show sufficient context to indicate the intended location.

The problem described in LWG [397^{\(i\)}](#) is still present: the streambuf operations can fail, and the sentry needs to handle that. The changes for LWG [835^{\(i\)}](#) ensure no exception is thrown if rdbuf()->pubsync() returns -1 on failure, but do nothing for the case where it throws an exception (the original topic of LWG 397!). Because C++11 made ~sentry implicitly noexcept, an exception from rdbuf()->pubsync() will terminate the process. That needs to be fixed.

Libstdc++ does terminate if pubsync() throws when called by ~sentry. Both MSVC and Libc++ silently swallow exceptions. It seems preferable to handle the exception and report an error, just as we do when pubsync() returns -1.

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 31.7.6.2.4 [\[ostream.sentry\]](#) as indicated:

```
~sentry();

-4- If (os.flags() & ios_base::unitbuf) && !uncaught_exceptions() && os.good() is true, calls os.rdbuf()-
>pubsync(). If that function returns -1 or exits via an exception, sets badbit in os.rdstate() without propagating an
exception.
```

[4200](#). The operation_state concept can be simplified

Section: 33.8.1 [\[exec.opstate.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Eric Niebler **Opened:** 2025-02-03 **Last modified:** 2025-02-07

Priority: Not Prioritized

Discussion:

Imported from [cplusplus/sender-receiver #312](#).

The current defn of the operation_state concept is:

```
template<class O>
concept operation_state =
    derived_from<typename O::operation_state_concept, operation_state_t> &&
    is_object_v<O> &&
    requires (O& o) {
        { start(o) } noexcept;
    };

```

I think the is_object_v<O> constraint is not needed because the derived_from constraint has already established that O is a class type.

And start(o) is always noexcept now that start mandates the noexcept-ness of op.start().

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 33.8.1 [\[exec.opstate.general\]](#) as indicated:

```
template<class O>
concept operation_state =
    derived_from<typename O::operation_state_concept, operation_state_t> &&
is_object_v<O> &&
    requires (O& o) {
{ start(o) } noexcept;
    };

```

4201. `with-await-transform::await_transform` should not use a deduced return type

Section: 33.9.4 [\[exec.awaitable\]](#) Status: [Tentatively Ready](#) Submitter: Brian Bi Opened: 2025-02-03 Last modified: 2025-02-07

Priority: Not Prioritized

Discussion:

Imported from [cplusplus/sender-receiver #309](#).

33.9.4 [\[exec.awaitable\]](#)/p5

The use of the deduced return type causes the definition of the sender's `as_awaitable` method to be instantiated too early, e.g., when the sender is passed to `get_completion_signatures`.

[Eric provides wording]

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 33.9.4 [\[exec.awaitable\]](#) as indicated:

-5- Let `with-await-transform` be the exposition-only class template:

```
namespace std::execution {
    template<class T, class Promise>
        concept has-as-awaitable = // exposition only
            requires (T&& t, Promise& p) {
                { std::forward<T>(t).as_awaitable(p) } -> is-awaitable<Promise&>;
            };

    template<class Derived>
        struct with-await-transform { // exposition only
            template<class T>
                T&& await_transform(T&& value) noexcept {
                    return std::forward<T>(value);
                }

            template<has-as-awaitable<Derived> T>
                decltype(auto) await_transform(T&& value)
                    noexcept(noexcept(std::forward<T>(value).as_awaitable(declval<Derived>())))
                    -> decltype(std::forward<T>(value).as_awaitable(declval<Derived>())) {
                    return std::forward<T>(value).as_awaitable(static_cast<Derived&>(*this));
                }
        };
}
```

4217. Clarify `mdspan` layout mapping requirements for `rank == 0`

Section: 23.7.3.4.2 [\[mdspan.layout.reqmts\]](#) Status: [Tentatively Ready](#) Submitter: Mark Hoemmen Opened: 2025-03-03 Last modified: 2025-03-09

Priority: Not Prioritized

Discussion:

23.7.3.4.2 [\[mdspan.layout.reqmts\]](#) p19-21 says that a layout mapping needs to provide `m.stride(r)`. However, 23.7.3.4.5.3 [\[mdspan.layout.left.obs\]](#) p5 constrains `layout_left::mapping<Extents>::stride(r)` on `Extents::rank() > 0` being true. The same is true of `layout_right::mapping` (23.7.3.4.6.3 [\[mdspan.layout.right.obs\]](#) p5). (The other Standard mappings in 23.7.3 [\[views.multidim\]](#) and 29.9 [\[linalg\]](#) do not have this constraint.) This suggests that a rank-zero `layout_{left,right}::mapping` does not conform with the layout mapping requirements.

On the other hand, other parts of 23.7.3.4.2 [\[mdspan.layout.reqmts\]](#) imply that `r` must be in the range `[0, rank())` for the layout mapping's extents type. If such an `r` does not exist, which is the case for a rank-zero layout mapping, then the `m.stride(r)` requirement is vacuous. This implies that a rank-zero `layout_{left,right}::mapping` fully conforms with the layout mapping requirements.

It is definitely the design intent for rank-zero `mdspan` to work, and for it to represent a view of a single element. Users can create rank-zero `mdspan` by invoking its constructor, or by using `submdspan` where all the slice arguments are convertible to `index_type`. Even though the normative wording permits this, adding a *Note* would clarify the design intent without making the wording redundant. This was the preferred change per LWG reflector discussion.

[2025-06-13; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 23.7.3.4.2 [[mdspan.layout.reqmts](#)] as indicated:

```
m.stride(r)
```

-19- *Preconditions*: `m.is_strided()` is true.

-20- *Result*: `typename M::index_type`

-21- *Returns*: `sr` as defined in `m.is_strided()` above.

[Note ?]: It is not required for `m.stride(r)` to be well-formed if `m.extents().rank()` is zero, even if `m.is_always_strided()` is true. — end note]

[4222](#). expected constructor from a single value missing a constraint

Section: 22.8.6.2 [[expected.object.cons](#)] **Status:** [Tentatively Ready](#) **Submitter:** Bronek Kozicki **Opened:** 2025-03-12 **Last modified:** 2025-03-15

Priority: Not Prioritized

View all other [issues](#) in [[expected.object.cons](#)].

Discussion:

When an expected object is initialized with a constructor taking first parameter of type `unexpected_t`, the expectation is that the object will be always initialized in disengaged state (i.e. the user expected postcondition is that `has_value()` will be false), as in the example:

```
struct T { explicit T(auto) {} };
struct E { E() {} };

int main() {
    expected<T, E> a(unexpected);
    assert(!a.has_value());
}
```

This does not hold when both value type `T` and error type `E` have certain properties. Observe:

```
struct T { explicit T(auto) {} };
struct E { E(int) {} }; // Only this line changed from the above example

int main() {
    expected<T, E> a(unexpected);
    assert(!a.has_value()); // This assert will now fail
}
```

In the example above the overload resolution of `a` finds the universal single parameter constructor for initializing expected in engaged state (22.8.6.2 [[expected.object.cons](#)] p23):

```
template<class U = remove_cv_t<T>>
constexpr explicit(!is_convertible_v<U, T>) expected(U&& v);
```

This constructor has a list of constraints which does not mention `unexpected_t` (but it mentions e.g. `unexpected` and `in_place_t`). Email exchange with the author of expected confirmed that it is an omission.

The proposed resolution is to add the following additional constraint to this constructor:

`is_same_v<remove_cvref_t<U>, unexpected_t> is false`

This will result in the above, most likely buggy, program to become ill-formed. If the user intent was for the object to be constructed in an engaged state, passing `unexpected_t` to the `T` constructor, they can fix the compilation error like so:

```
expected<T, E> a(in_place, unexpected);
```

[2025-06-13; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 22.8.6.2 [[expected.object.cons](#)] as indicated:

```
template<class U = remove_cv_t<T>>
constexpr explicit(!is_convertible_v<U, T>) expected(U&& v);
```

-23- *Constraints*:

(23.1) — `is_same_v<remove_cvref_t<U>, in_place_t>` is false; and

(23.2) — `is_same_v<expected, remove_cvref_t<U>>` is false; and

(23.?) — `is_same_v<remove_cvref_t<U>, unexpect_t>` is false; and

(23.3) — `remove_cvref_t<U>` is not a specialization of `unexpected`; and

(23.4) — `is_constructible_v<T, U>` is true; and

(23.5) — if `T` is `cv bool`, `remove_cvref_t<U>` is not a specialization of `expected`.

-24- *Effects*: Direct-non-list-initializes `val` with `std::forward<U>(v)`.

-25- *Postconditions*: `has_value()` is true.

-26- *Throws*: Any exception thrown by the initialization of `val`.

4224. Philox engines should be freestanding

Section: 29.5.4.5 [[rand.eng.philox](#)] **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2025-03-15 **Last modified:** 2025-03-15

Priority: Not Prioritized

View other [active issues](#) in [[rand.eng.philox](#)].

View all other [issues](#) in [[rand.eng.philox](#)].

Discussion:

Philox engines don't seem to require floating-point operations or support from the operating system, so they are probably suitable for freestanding. However, as [P2976R1](#) was finished before the adoption of [P2075R6](#), these engines are not made freestanding yet.

[2025-06-13; Reflector poll]

Set status to Tentatively Ready after ten votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 29.5.2 [[rand.synopsis](#)], header `<random>` synopsis, as indicated:

```
[...]
// 29.5.4.5 [rand.eng.philox], class template philox_engine
template<class UIntType, size_t w, size_t n, size_t r, UIntType... consts>
    class philox_engine;                // partially freestanding

[...]
```

```
[...]
using philox4x32 = see below; // freestanding
using philox4x64 = see below; // freestanding
[...]
```

2. Modify 29.5.4.5 [[rand.eng.philox](#)], class template `philox_engine` synopsis, as indicated:

```
namespace std {
    template<class UIntType, size_t w, size_t n, size_t r, UIntType... consts>
        class philox_engine {
            [...]
            // inserters and extractors
            template<class charT, class traits>
                friend basic_ostream<charT, traits>&
                    operator<<(basic_ostream<charT, traits>& os, const philox_engine& x); // hosted
            template<class charT, class traits>
                friend basic_istream<charT, traits>&
                    operator>>(basic_istream<charT, traits>& is, philox_engine& x); // hosted
        };
}
```

4227. Missing `noexcept` operator in `[exec.when.all]`

Section: 33.9.12.11 [[exec.when.all](#)] **Status:** [Tentatively Ready](#) **Submitter:** Ian Petersen **Opened:** 2025-03-17 **Last modified:** 2025-03-22

Priority: Not Prioritized

Discussion:

In 33.9.12.11 [\[exec.when.all\]](#) p7, the *impls-for*<when_all_t>::*get-state* member is defined to be equivalent to the following lambda:

```
[<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(e) -> decltype(e) {  
    return e;  
}
```

and *e* is later defined to be:

```
std::forward<Sndr>(sndr).apply(make-state<Rcvr>())
```

Together, the two definitions imply that the *noexcept* clause on the provided lambda is:

```
noexcept(std::forward<Sndr>(sndr).apply(make-state<Rcvr>()))
```

which is invalid.

Presumably, the lambda should be defined like so (with an extra *noexcept* operator in the *noexcept* clause):

```
[<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(noexcept(e)) -> decltype(e) {  
    return e;  
}
```

[2025-06-13; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 33.9.12.11 [\[exec.when.all\]](#) as indicated:

-7- The member *impls-for*<when_all_t>::*get-state* is initialized with a callable object equivalent to the following lambda expression:

```
[<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(noexcept(e)) -> decltype(e) {  
    return e;  
}
```

4231. `datapar::chunk<N>` should use *simd-size-type* instead of `size_t`

Section: 29.10.3 [\[simd.syn\]](#), 29.10.7.8 [\[simd.creation\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Matthias Kretz **Opened:** 2025-03-22 **Last modified:** 2025-03-22

Priority: Not Prioritized

Discussion:

All index values and *simd* widths in subclause "Data-parallel types" use the type *simd-size-type*. Specifically, the NTTP of `std::datapar::resize` uses *simd-size-type* and `std::datapar::chunk` is "implemented" using `std::datapar::resize`.

For `chunk<N>`, *N* is of type `size_t` and needs to be converted to *simd-size-type* in the effects clause where it is passed to `resize`. The NTTP of `chunk` should use *simd-size-type* instead of `size_t`.

[2025-06-13; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 29.10.3 [\[simd.syn\]](#), header <simd> synopsis, as indicated:

```
namespace std::datapar {  
    [...]   
    template<simd-size-typesize_t N, class T, class Abi>  
        constexpr auto chunk(const basic_simd<T, Abi>& x) noexcept;  
    template<simd-size-typesize_t N, size_t Bytes, class Abi>  
        constexpr auto chunk(const basic_simd_mask<Bytes, Abi>& x) noexcept;  
    [...]   
}
```

2. Modify 29.10.7.8 [\[simd.creation\]](#) as indicated:

```
template<simd-size-typesize_t N, class T, class Abi>  
    constexpr auto chunk(const basic_simd<T, Abi>& x) noexcept;
```

-4- *Effects*: Equivalent to: `return chunk<resize_t<N, basic_simd<T, Abi>>>(x);`

```
template<simd-size-type size_t N, size_t Bytes, class Abi>
constexpr auto chunk(const basic_simd_mask<Bytes, Abi>& x) noexcept;
```

-5- *Effects*: Equivalent to: `return chunk<resize_t<N, basic_simd_mask<Bytes, Abi>>>(x);`

[4232](#). `datapar::resize` does not resize

Section: 29.10.4 [\[simd.traits\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2025-03-24 **Last modified:** 2025-06-12

Priority: Not Prioritized

Discussion:

The wording actually requires the size to be left unchanged.

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 29.10.4 [\[simd.traits\]](#) as indicated:

```
template<simd-size-type N, class V> struct resize { using type = see below; };
```

[...]

-9- If `V` is a specialization of `basic_simd`, let `Abi1` denote an ABI tag such that `basic_simd<T, Abi1>::size()` equals `V::size()`~~`N`~~. If `V` is a specialization of `basic_simd_mask`, let `Abi1` denote an ABI tag such that `basic_simd_mask<sizeof(T), Abi1>::size()` equals `V::size()`~~`N`~~.

[4233](#). The helper lambda of `std::erase` for hive should specify return type as `bool`

Section: 23.3.9.6 [\[hive.erase\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2025-03-24 **Last modified:** 2025-06-12

Priority: Not Prioritized

Discussion:

This is a follow up to LWG [4135](#)⁽¹⁾, which incidentally adds a default template parameter for `U` to be consistent with other erase functions, which is editorial since the declaration already has it.

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 23.3.9.6 [\[hive.erase\]](#) as indicated:

```
template<class T, class Allocator, class U = T>
typename hive<T, Allocator>::size_type
erase(hive<T, Allocator>& c, const U& value);
```

-1- *Effects*: Equivalent to:

```
return erase_if(c, [&](const auto& elem) -> bool { return elem == value; });
```

[4234](#). Including `<hive>` doesn't provide `std::begin/end`

Section: 24.7 [\[iterator.range\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2025-03-25 **Last modified:** 2025-06-12

Priority: Not Prioritized

View other [active issues](#) in [iterator.range].

View all other [issues](#) in [iterator.range].

Discussion:

24.7 [\[iterator.range\]](#) should add `<hive>` to the list as the latter provides a series of range access member functions such as `begin/end`.

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 24.7 [\[iterator.range\]](#) as indicated:

-1- In addition to being available via inclusion of the `<iterator>` header, the function templates in 24.7 [\[iterator.range\]](#) are available when any of the following headers are included: `<array>`, `<deque>`, `<flat_map>`, `<flat_set>`, `<forward_list>`, `<hive>`, `<inplace_vector>`, `<list>`, `<map>`, `<regex>`, `<set>`, `<span>`, `<string>`, `<string_view>`, `<unordered_map>`, `<unordered_set>`, and `<vector>`.

4235. `cache_latest_view` and `to_input_view` miss `reserve_hint`

Section: 25.7.34.2 [\[range.cache.latest.view\]](#), 25.7.35.2 [\[range.to.input.view\]](#) Status: [Tentatively Ready](#) Submitter: Hewill Kang Opened: 2025-03-25
Last modified: 2025-06-12

Priority: 2

Discussion:

Intuitively, both view classes should also have `reserve_hint` members.

[2025-06-12; Reflector poll]

Set priority to 2 after reflector poll.

[2025-06-13; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 25.7.34.2 [\[range.cache.latest.view\]](#) as indicated:

```
namespace std::ranges {
    template<input_range V>
        requires view<V>
        class cache_latest_view : public view_interface<cache_latest_view<V>> {
            [...]
            constexpr auto size() requires sized_range<V>;
            constexpr auto size() const requires sized_range<const V>;

            constexpr auto reserve_hint() requires approximately_sized_range<V>;
            constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
        };
    [...]
}
```

[...]

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

-4- Effects: Equivalent to: `return ranges::size(base_);`

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
```

-?- Effects: Equivalent to: `return ranges::reserve_hint(base_);`

2. Modify 25.7.35.2 [\[range.to.input.view\]](#) as indicated:

```
template<input_range V>
    requires view<V>
    class to_input_view : public view_interface<to_input_view<V>> {
        [...]
        constexpr auto size() requires sized_range<V>;
        constexpr auto size() const requires sized_range<const V>;
```

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
};
[...]
```

[...]

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

-5- *Effects*: Equivalent to: `return ranges::size(base_);`

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
```

-?- *Effects*: Equivalent to: `return ranges::reserve_hint(base_);`

4236. `chunk_view::outer_iterator::value_type` should provide `reserve_hint`

Section: 25.7.29.4 [[range.chunk.outer.value](#)] **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2025-03-26 **Last modified:** 2025-06-12

Priority: Not Prioritized

View other [active issues](#) in [[range.chunk.outer.value](#)].

View all other [issues](#) in [[range.chunk.outer.value](#)].

Discussion:

Consider:

```
views::istream<int>(is) | views::chunk(N) | ranges::to<std::list<std::vector<int>>>();
```

When the stream is large enough, each chunk will be of size N in most cases, except it is the last chunk.

In this case, there is no reason not to provide a `reserve_hint` as this can be easily done by just return `remainder_`. Otherwise, when N is large, each vector will be reallocated multiple times unnecessarily.

This is also consistent with the `forward_range` version, since its value type is `views::take(subrange(current_, end_), n_)`, which always has a `reserve_hint` as `take_view` unconditionally provides it.

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 25.7.29.4 [[range.chunk.outer.value](#)] as indicated:

```
namespace std::ranges {
    template<view V>
        requires input_range<V>
        struct chunk_view<V>::outer_iterator::value_type : view_interface<value_type> {
            [...]
            constexpr auto size() const
                requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;

            constexpr auto reserve_hint() const noexcept;
        };
}
```

[...]

```
constexpr auto size() const
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

-4- *Effects*: Equivalent to:

```
return to-unsigned-like(ranges::min(parent_>remainder_,
                                     ranges::end(parent_>base_) - *parent_>current_));
```

```
constexpr auto reserve_hint() const noexcept;
```

-?- *Effects*: Equivalent to:

```
return to-unsigned-like(parent_>remainder_);
```

4239. flat_map's transparent comparator no longer works for string literals

Section: 23.6.8.7 [\[flat.map.modifiers\]](#) **Status:** [New](#) **Submitter:** Hui Xie **Opened:** 2025-03-29 **Last modified:** 2025-03-29

Priority: Not Prioritized

View other [active issues](#) in [\[flat.map.modifiers\]](#).

View all other [issues](#) in [\[flat.map.modifiers\]](#).

Discussion:

According to the spec, the following code should hard error

```
std::flat_map<std::string, int, std::less<>> m;
m.try_emplace("abc", 5); // hard error
```

The reason is that we specify in 23.6.8.7 [\[flat.map.modifiers\]](#) p21 the effect to be as if `ranges::upper_bound` is called.

`ranges::upper_bound` requires `indirect_strict_weak_order`, which requires the comparator to be invocable for all combinations. In this case, it requires

```
const char (&)[4] < const char (&)[4]
```

to be well-formed, which is no longer the case in C++26 after [P2865R6](#).

We should just use `std::upper_bound` instead. `libstdc++` already uses `std::upper_bound`. `libc++` uses `ranges::upper_bound` but `clang` has not yet implemented [P2865](#) properly.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 23.6.8.7 [\[flat.map.modifiers\]](#) as indicated:

```
template<class K, class... Args>
constexpr pair<iterator, bool> try_emplace(K&& k, Args&&... args);
template<class K, class... Args>
constexpr iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

-19- *Constraints:* [...]

-20- *Preconditions:* [...]

-21- *Effects:* If the map already contains an element whose key is equivalent to `k`, `*this` and `args...` are unchanged. Otherwise equivalent to:

```
auto key_it = upper_bound(c.keys.begin(), c.keys.end(), k, compare); ranges::upper_bound(c.keys, k, compare);
auto value_it = c.values.begin() + distance(c.keys.begin(), key_it);
c.keys.emplace(key_it, std::forward<K>(k));
c.values.emplace(value_it, std::forward<Args>(args)...);
```

4242. ranges::distance does not work with volatile iterators

Section: 24.4.4.3 [\[range.iter.op.distance\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2025-04-12 **Last modified:** 2025-06-12

Priority: Not Prioritized

View all other [issues](#) in [\[range.iter.op.distance\]](#).

Discussion:

After LWG [3664^{\(i\)}](#), `ranges::distance` computes the distance between `last` and `first` by returning `last - static_cast<const decay_t<I>&>(first)` when the two are subtractable. However, this will cause a hard error if `first` is volatile-qualified ([demo](#)):

```
#include <iterator>

int main() {
    int arr[] = {1, 2, 3};
    int* volatile ptr = arr;
    // return std::distance(ptr, arr + 3); // this is ok
    return std::ranges::distance(ptr, arr + 3); // hard error
}
```

The resolution changes the *Effects* of LWG [3664^{\(i\)}](#) from "cute" to "noncute".

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

"Would prefer `auto(first)` but the current wording is correct."

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 24.4.4.3 [\[range.iter.op.distance\]](#) as indicated:

```
template<class I, sized_sentinel_for<decay_t<I>> S>
constexpr iter_difference_t<decay_t<I>> ranges::distance(I&& first, S last);

-3- Effects: Equivalent to:

if constexpr (!is_array_v<remove_reference_t<I>>>)
    return last - first;
else
    return last - static_cast<const decay_t<I>&>(first);
```

4245. Operators that interact with `counted_iterator` and `default_sentinel_t` should be `noexcept`

Section: 24.5.7.1 [\[counted_iterator\]](#), 24.5.7.5 [\[counted.iter.nav\]](#), 24.5.7.6 [\[counted.iter.cmp\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2025-04-18 **Last modified:** 2025-06-12

Priority: Not Prioritized

View all other [issues](#) in `[counted.iterator]`.

Discussion:

`counted_iterator` can be compared or subtracted from `default_sentinel_t`, which only involves simple integer arithmetic and does not have any *Preconditions*.

In this case, it is reasonable to declare them as `noexcept`.

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 24.5.7.1 [\[counted_iterator\]](#) as indicated:

```
namespace std {
    template<input_or_output_iterator I>
    class counted_iterator {
    public:
        [...]
        friend constexpr iter_difference_t<I> operator-(
            const counted_iterator& x, default_sentinel_t) noexcept;
        friend constexpr iter_difference_t<I> operator-(
            default_sentinel_t, const counted_iterator& y) noexcept;
        [...]
        friend constexpr bool operator==(
            const counted_iterator& x, default_sentinel_t) noexcept;
        [...]
    };
    [...]
}
```

2. Modify 24.5.7.5 [\[counted.iter.nav\]](#) as indicated:

```
friend constexpr iter_difference_t<I> operator-(
    const counted_iterator& x, default_sentinel_t) noexcept;

-15- Effects: Equivalent to: return -x.length;

friend constexpr iter_difference_t<I> operator-(
    default_sentinel_t, const counted_iterator& y) noexcept;

-16- Effects: Equivalent to: return y.length;
```

3. Modify 24.5.7.6 [\[counted.iter.cmp\]](#) as indicated:

```
friend constexpr bool operator==(
    const counted_iterator& x, default_sentinel_t) noexcept;
```

-3- *Effects*: Equivalent to: return x.length == 0;

4247. Header <stdbit.h> is not yet freestanding

Section: 22.12 [[stdbit.h.syn](#)] **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2025-04-24 **Last modified:** 2025-06-12

Priority: Not Prioritized

Discussion:

Per C23/WG14 [N3220](#) 4 p7, <stdbit.h> is freestanding in C23, but C++ hasn't required it for a freestanding implementation. LWG [4049^{\(i\)}](#) is related but doesn't cover this, because there's no <cstdbit> standard header.

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

- 1. Modify Table 27 [tab:headers.cpp.fs] as indicated:

Table 27: C++ headers for freestanding implementations
[tab:headers.cpp.fs]

	Subclause	Header
[...]		
22.11 [bit]	Bit manipulation	<bit>
22.12 [stdbit.h.syn]	C-compatible bit manipulation	<stdbit.h>
23.3.3 [array]	Class template array	<array>
[...]		

- 2. Modify 22.12 [[stdbit.h.syn](#)], header <stdbit.h> synopsis, as indicated:

```
// all freestanding
#define __STDC_VERSION__ STDBIT_H__ 202311L

#define __STDC_ENDIAN_BIG__ see below
#define __STDC_ENDIAN_LITTLE__ see below
#define __STDC_ENDIAN_NATIVE__ see below

[...]
```